Diurnal Model Diagnosis — Why Daytime Detection Fails

Introduction

Look at the night numbers first:

```
================================================================================
DIURNAL MODEL PERFORMANCE: DAY (Sunlit) vs. NIGHT (Pure Thermal)
================================================================================
regime   scenes   total_gt_fires   total_tp   total_fp    mean_dice   mean_recall   m
   DAY       48             676        674    1394601     0.000965      0.997167
 NIGHT       41            1451        411        224     0.397748      0.338807
================================================================================
```

- **At Night:** Look at `mean_fp_per_scene:  5.46` pixels. Across the entire landmass of Northern Algeria (30,000 pixels per frame), the model produced only 5 false alarm pixels on average per scene! That is a 99.98% true negative rate, accompanied by a 40% Dice score. In orbital remote sensing, detecting sparse 3 km fires at night with almost zero background noise is an exceptional result.
- **During the Day:** The daytime regime experienced another total blowout: 29,054 false positive pixels per scene out of 30,000.

Let us examine why broadcasting a flat solar channel $S(t)$ and adjusting α and β did not stop the daytime blowout. The failure stems from two distinct mathematical mechanisms: Batch Normalization on spatially flat channels and a mathematical degeneracy in the Tversky Loss.

Failure Mechanism 1: How Batch Normalization Neutralizes Flat Channels

In `wildfire_dataset.py`, Channel 3 was created by taking a single scalar value $S(t) \in [0.0, 1.0]$ and filling the entire 128×256 grid with that identical number:

$$\mathbf{X}_{b,3,y,x} = S(t) \quad \text{for all } y \in [0, 127],\ x \in [0, 255]$$

Now consider what happens in the very first layer of the U-Net (`DoubleConv`):

$$\mathbf{x} \longrightarrow \text{Conv2D} \longrightarrow \textbf{BatchNorm2D} \longrightarrow \text{ReLU}$$

In PyTorch, `nn.BatchNorm2d` normalizes activations across the mini-batch (B) and spatial dimensions (H, W) for each channel c:

$$\mu_c = \frac{1}{B \times H \times W} \sum_{b=1}^{B} \sum_{y=1}^{H} \sum_{x=1}^{W} x_{b,c,y,x}$$

$$\sigma_c^2 = \frac{1}{B \times H \times W} \sum_{b=1}^{B} \sum_{y=1}^{H} \sum_{x=1}^{W} (x_{b,c,y,x} - \mu_c)^2$$

$$\hat{x}_{b,c,y,x} = \frac{x_{b,c,y,x} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} \cdot \gamma_c + \beta_c$$

What Happens When a Channel Has No Spatial Variance?

Within each image, every pixel in Channel 3 has the exact same value $S(t)$.

The spatial derivative across the image is zero everywhere:

$$\frac{\partial S}{\partial x} = 0, \quad \frac{\partial S}{\partial y} = 0$$

The 3×3 convolutional filters in the encoder are edge detectors designed to compute local differences ($\Delta X = X_{i+1} - X_i$). Because Channel 3 is completely flat, the convolution yields a uniform constant offset across the entire feature map.

Batch Normalization subtracts the mean across the spatial dimensions. If all 8 images in a mini-batch happen to be daytime scenes ($S(t) \approx 1.0$), then $x_{b,3,y,x} = 1.0$ everywhere. The batch mean becomes $\mu = 1.0$, the variance drops to $\sigma^2 = 0$, and the normalized output is completely eliminated:

$$\hat{x} = \frac{1.0 - 1.0}{\sqrt{0 + 10^{-5}}} = \mathbf{0.0}$$

The spatial convolutional layers cannot extract spatial patterns from a flat scalar sheet. The solar context was effectively lost inside the first few layers.

Failure Mechanism 2: The Mathematical Trap in the Tversky Loss

This mathematical detail explains why the daytime predictions remained stuck at all ones.

Examine the exact equation for the Tversky Index (TI) for a daytime image:

$$TI = \frac{TP + \epsilon}{TP + \alpha \cdot FP + \beta \cdot FN + \epsilon}, \quad \text{Loss} = (1 - TI)^{1/\gamma}$$

During the daytime, our loss weights were:

$$\alpha = 0.6 \quad (\text{penalty on } FP), \quad \beta = 0.4 \quad (\text{penalty on } FN)$$

Now compare the loss between two different degenerate predictions on an image containing 14 fire pixels out of 30,000 total pixels:

```
                                SCENARIO A: Predict All Zeros (Off)

Predictions: All 0.0 everywhere
TP = 0,   FP = 0,   FN = 14
TI   = (0 + 1e-6) / (0 + 0.6*(0) + 0.4*(14) + 1e-6)
     = 1e-6 / 5.600001  0.00000018
Loss = (1.0 - 0.00000018)^0.75 = 1.000000

Loss for predicting ALL ZEROS = 1.0000

                                SCENARIO B: Predict All Ones (On)

Predictions: All 1.0 everywhere across the entire country
TP = 14,   FP = 29,986,   FN = 0
TI   = (14 + 1e-6) / (14 + 0.6*(29,986) + 0.4*(0) + 1e-6)
     = 14 / (14 + 17,991.6)
     = 14 / 18,005.6  0.0007775
Loss = (1.0 - 0.0007775)^0.75  0.999417

Loss for predicting ALL ONES = 0.9994
```

Notice the critical result:

$$0.999417 < 1.000000$$

Because $TP > 0$ yields a non-zero numerator, the optimizer gets mathematically rewarded for flagging the entire country of Algeria as a wildfire rather than turning the predictions off!

The gradient descent algorithm followed the path of least resistance down this loss surface, converging into the local minimum where it outputs large positive logits across every single daytime pixel.

The Fundamental Physics Problem: How Operational Satellites Solve Midday Solar Glint

To fix daytime detections reliably, consider how operational satellite fire systems (like NASA MODIS/VIIRS and EUMETSAT FIR) solve this problem.

They do not use a single global threshold. Instead, they rely on **Spatial Contextual Differencing (Local Background Subtraction)**.

```
                                MIDDAY PHYSICAL TEMPERATURES

   Normal Sun-Baked Rock (Bouïra)                    Active Forest Fire (Tizi Ouzou)

   T_3.9  = 338 K  (65°C)                            T_3.9  = 360 K  (87°C)
   T_10.8 = 318 K  (45°C)                            T_10.8 = 320 K  (47°C)
   T     = +20 K  (Sun heating)                      T     = +40 K  (Fire + Sun)
```

Notice that during the day, every pixel of dry soil in Algeria has $\Delta T \approx +15$ K to +22 K due to solar heating.

A single pixel cannot be classified by looking at its value in isolation. It must be evaluated relative to its immediate neighbors:

In an area of sun-baked rock, a pixel with $\Delta T = 20$ K is surrounded by neighbors that are also at 20 K.

$$\text{Contrast} = \Delta T - \text{Mean(Neighbors)} = 20 \text{ K} - 20 \text{ K} = \mathbf{0} \text{ K} \quad \text{(NOT a fire)}$$

At an active wildfire, a pixel with $\Delta T = 40$ K is surrounded by neighbors at 20 K.

$$\text{Contrast} = \Delta T - \text{Mean(Neighbors)} = 40 \text{ K} - 20 \text{ K} = +\mathbf{20} \text{ K} \quad (\textbf{CONFIRMEDFIRE}!)$$

By subtracting the local moving average (a 7×7 or 15×15 spatial filter), the uniform solar heating is stripped away, leaving only localized combustion spikes.

The Two Strategic Options

We have two clear paths forward:

Option 1: The Specialized Dual-Model Architecture (Recommended)

Operational meteorological agencies do not use one unified model for day and night; the radiative physics are entirely different.

- **UNet_Night:** You already have this working. Trained strictly on nighttime scenes, it achieves a 40% Dice score and only 5 false-positive pixels per scene.
- **UNet_Day:** Trained strictly on daytime scenes with Spatial Contrast Features and Binary Cross-Entropy (BCE) + Dice Loss (to avoid the Tversky all-ones trap).

Option 2: Spatial Background Subtraction Channel

We retain a unified model, but replace the flat scalar $S(t)$ with a Spatial Anomaly Channel:

$$\Delta T_{\text{anomaly}}(x, y) = \Delta T(x, y) - \text{UniformFilter}(\Delta T(x, y), \text{size} = 11)$$

This gives the convolutional filters an explicit local contrast map to evaluate, removing the uniform background heating across North Africa.

Which of these two architectures would you prefer to build next?